

Assignment 3: Unsupervised Learning — Vancouver Business Licences Explorer

Setup

Importing the necessary files for this assignment.

```
In [31]: # Import core and specialist libraries
import json
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn

# Preprocessing, metrics, clustering, PCA, t-SNE, and UMAP
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import KMeans, DBSCAN
import hdbscan
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.utils import resample
import umap

# Configure plotting styles and disable warnings
warnings.filterwarnings("ignore")
sns.set_style("whitegrid")
plt.rcParams["figure.dpi"] = 100
RANDOM_STATE = 42

print("Setup complete.")
```

Setup complete.

A1 - Data Acquisition & Cleaning

Exploration: Data Schema

The following table is the first value retrieved from the geojson file. Formatted and explored via mapshaper.org -- command used:

Console:

-info

Field	Value
businessname	(Anie Philip)
businesssubtype	
businessstradename	
businessstype	Long-term Rental
city	Vancouver
country	CA
expireddate	2025-12-31
extractdate	2026-07-01T02:32:20-07:00
feepaid	94
folderyear	25
geo_point_2d	
geom	
house	4959
issueddate	2024-11-25T21:02:01-08:00
licencenumber	25-129785
licencerevisionnumber	00
licencersn	4637506
localarea	Renfrew-Collingwood
numberofemployees	0
postalcode	V5R 3T5
province	BC
status	Inactive
street	Moss Street
unit	
unittype	

Cleaning: Creating Data Subset

The parent geojson file is ~140mb, which makes loading and processing difficult when attempting to work with it. I've performed the following operations to shrink 140mb -> 32mb at mapshaper using the following commands:

First, to drop unnecessary fields (justifications below):

-filter-fields

businessname,businesstradename,geo_point_2d,localarea,postalcode,busin

Then, drop entries that don't match our conditions:

-filter "status == 'Issued' && geo_point_2d != null"

Dropped:

Fields:

- country & province (They are all going to be "CA" and "BC")
- extractdate (Just a system download timestamp)
- folderyear (Not really relevant for any features)
- geom & house & street & unit & unittype (we already have geo_point_2d for exact mapping; don't need text addresses)

Conditions:

- status must be "Issued"
- geo_point_2d must NOT be null

Cleaning: Updated Data Schema (Example)

Field	Value
businessname	Anie Philip
businesssubtype	
businesstradename	
businesstype	Long-term Rental
expireddate	2025-12-31
feepaid	94
geo_point_2d	{...}
issueddate	2024-11-25 (21:02:01 -08:00)
licencenumber	25-129785
licencersn	4637506
licenserevisionnumber	00
localarea	Renfrew-Collingwood
numberofemployees	0
postalcode	V5R 3T5
status	Inactive

```
In [ ]: # open file "business_locations.geojson"
# load geojson_data using json parser
# print total features count
# print formatted JSON representation of the first feature

with open("data/business-licences-curated-v3.geojson", "r") as file:
    _data = file.read()

# Parse
_data = json.loads(_data)
print(f"Total Features: {len(_data["features"])}")

data = json.dumps(_data, indent=2)
```

Total Features: 85796

Cleaning: Convert to Dataframe

```
In [33]: rows = []
columns=[
    "businessname",
    "businesssubtype",
    "businesstype",
    "businessstradename",
    "expireddate",
    "feepaid",
    "issueddate",
    "licencenumber",
    "licencerevisionnumber",
    "licencersn",
    "Longitude",
    "Latitude",
    "localarea",
    "numberofemployees",
    "postalcode",
    "status"
]
for feat in _data["features"]:
    props = feat["properties"]
    longitude, latitude = feat["geometry"]["coordinates"]
    rows.append({**props, "Longitude": longitude, "Latitude": latitude})
df = pd.DataFrame(rows, columns=columns)

df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 85796 entries, 0 to 85795
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   businessname                          85793 non-null   str
1   businesssubtype                       16200 non-null   str
2   businesstype                           85796 non-null   str
3   businesstradename                     42895 non-null   str
4   expireddate                           85651 non-null   str
5   feepaid                               59332 non-null   float64
6   issueddate                            85652 non-null   str
7   licencenumber                         85796 non-null   str
8   licencerevisionnumber                 85796 non-null   str
9   licencersn                            85796 non-null   str
10  Longitude                             85796 non-null   float64
11  Latitude                             85796 non-null   float64
12  localarea                             85796 non-null   str
13  numberofemployees                     85796 non-null   int64
14  postalcode                            85142 non-null   str
15  status                                85796 non-null   str
dtypes: float64(3), int64(1), str(12)
memory usage: 21.9 MB
```

```
In [34]: print("Average Missing Values Per Column:")
for col in columns:
    print(f"--- {col}: {df[col].isna().mean()}")
```

```
Average Missing Values Per Column:
--- businessname: 3.4966665112592664e-05
--- businesssubtype: 0.8111800083919997
--- businesstype: 0.0
--- businesstradename: 0.5000349666651126
--- expireddate: 0.0016900554804419787
--- feepaid: 0.3084526085132174
--- issueddate: 0.0016783999254044479
--- licencenumber: 0.0
--- licencerevisionnumber: 0.0
--- licencersn: 0.0
--- Longitude: 0.0
--- Latitude: 0.0
--- localarea: 0.0
--- numberofemployees: 0.0
--- postalcode: 0.007622732994545201
--- status: 0.0
```

Cleaning: Deriving a Fine-Grained Industry Label

`businesstype` is always present, but `businesssubtype` gives a more specific label where it exists (~19% of rows). Combine the two into a single `industry` column:

"`businesstype – businesssubtype`" where a subtype exists, falling back to `businesstype` alone otherwise.

```
In [35]: df["industry"] = np.where(
    df["businesssubtype"].notna(),
    df["businesstype"] + " - " + df["businesssubtype"],
    df["businesstype"]
)

print(f"Unique businesstype values: {df['businesstype'].nunique()}")
print(f"Unique industry values: {df['industry'].nunique()}")
df[["businesstype", "businesssubtype", "industry"]].sample(10, random_state=
```

Unique businesstype values: 90
Unique industry values: 156

```
Out[35]:
```

	businesstype	businesssubtype	industry
24215	Long-term Rental	NaN	Long-term Rental
72678	Marina Operator	Live Aboard	Marina Operator - Live Aboard
50447	Retail Dealer	NaN	Retail Dealer
67229	Design Services	NaN	Design Services
35073	Laundry Services	NaN	Laundry Services
18215	Health Care Professionals and Services	NaN	Health Care Professionals and Services
12325	Legal Services	NaN	Legal Services
80513	Retail Dealer - Food	NaN	Retail Dealer - Food
53232	Long-term Rental	NaN	Long-term Rental
18278	Health Care Professionals and Services	NaN	Health Care Professionals and Services

Cleaning: Identifying Duplicates

```
In [36]: # Dry run/sanity check of baseline
df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 85796 entries, 0 to 85795
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   businessname                          85793 non-null  str
1   businesssubtype                       16200 non-null  str
2   businesstype                          85796 non-null  str
3   businessstradename                   42895 non-null  str
4   expireddate                          85651 non-null  str
5   feepaid                              59332 non-null  float64
6   issueddate                          85652 non-null  str
7   licencenumber                        85796 non-null  str
8   licencerevisionnumber                85796 non-null  str
9   licencersn                           85796 non-null  str
10  Longitude                             85796 non-null  float64
11  Latitude                             85796 non-null  float64
12  localarea                            85796 non-null  str
13  numberofemployees                    85796 non-null  int64
14  postalcode                           85142 non-null  str
15  status                               85796 non-null  str
16  industry                             85796 non-null  str
dtypes: float64(3), int64(1), str(13)
memory usage: 24.8 MB

```

```

In [37]: # Returns a True/False series for every row
duplicate_mask = df.duplicated()

# Filter the DataFrame to see the duplicate entries
duplicate_rows = df[duplicate_mask]

duplicate_rows

```

```

Out[37]:
   businessname  businesssubtype  businesstype  businessstradename  expireddate
79427      IMX PORT TRADING INC.      NaN      Retail Dealer      Ollie & Co.  2026-12-31

```

Only one duplicated entry in this row. Nice!

```

In [38]: df = df.drop_duplicates()
df.info()

```

```

<class 'pandas.DataFrame'>
Index: 85795 entries, 0 to 85795
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   businessname                          85792 non-null  str
1   businesssubtype                       16200 non-null  str
2   businesstype                           85795 non-null  str
3   businesstradename                     42894 non-null  str
4   expireddate                           85650 non-null  str
5   feepaid                               59331 non-null  float64
6   issuedate                             85651 non-null  str
7   licencenumber                         85795 non-null  str
8   licencerevisionnumber                 85795 non-null  str
9   licensorsn                           85795 non-null  str
10  Longitude                             85795 non-null  float64
11  Latitude                             85795 non-null  float64
12  localarea                             85795 non-null  str
13  numberofemployees                     85795 non-null  int64
14  postalcode                            85141 non-null  str
15  status                                85795 non-null  str
16  industry                              85795 non-null  str
dtypes: float64(3), int64(1), str(13)
memory usage: 25.5 MB

```

Note - Using LicenseRSN as ID

Referring to the following: <https://support.safe.com/hc/en-us/articles/25407521475341-How-to-Prepare-Data-for-Tableau-with-FME-Processing-Spreadsheets>

LicenseNumber is an ID number that can be reused and may therefore not be unique. Also, there may be multiple records for the same LicenseNumber; for example, a single business that operates at multiple addresses will have a record in the dataset for each address (but with the same LicenseNumber). LicenseRSN is a unique ID number per license; i.e. each license issued should have a unique LicenseRSN. Of course, there may be multiple records with this value as well, because it should match each record of a multiple LicenseNumber feature.

Following the previously established context, I will be using LicenseRSN as the unique identifier.

A1 - Write up - Cleaning Justifications

The A1 cleaning pipeline made several targeted decisions to shrink and simplify the raw business-licences dataset without discarding anything analytically useful. First, the ~140MB source GeoJSON was pre-processed in mapshaper to drop nine fields: country and province (constant across all rows, since every record is Vancouver, BC, CA),

extractdate and folderyear (system metadata irrelevant to any planned feature), and geom, house, street, unit, and unittype (redundant once geo_point_2d provides exact coordinates, making the text address unnecessary) thereby reducing the file to 32MB before it ever hit pandas, which kept load times manageable. Rows were also filtered at this stage to status == 'Issued' with a non-null geo_point_2d, since inactive/expired licences and records lacking coordinates can't support the geographic clustering planned in A2. After loading into a DataFrame, businesstype and businesssubtype were combined into a single industry column (falling back to businesstype alone when no subtype exists), preserving the ~19% of rows with more specific subtype information rather than discarding it. A duplicated() check found exactly one true duplicate row, which was dropped. Finally, rather than using licencenumber as a unique identifier, licencersn was chosen instead, justified by external documentation noting that licence numbers can be reused or shared across multiple records (e.g., one business at multiple addresses), whereas licencersn is guaranteed unique per licence record, thus making it the more reliable key for any row-level operations later in the analysis.

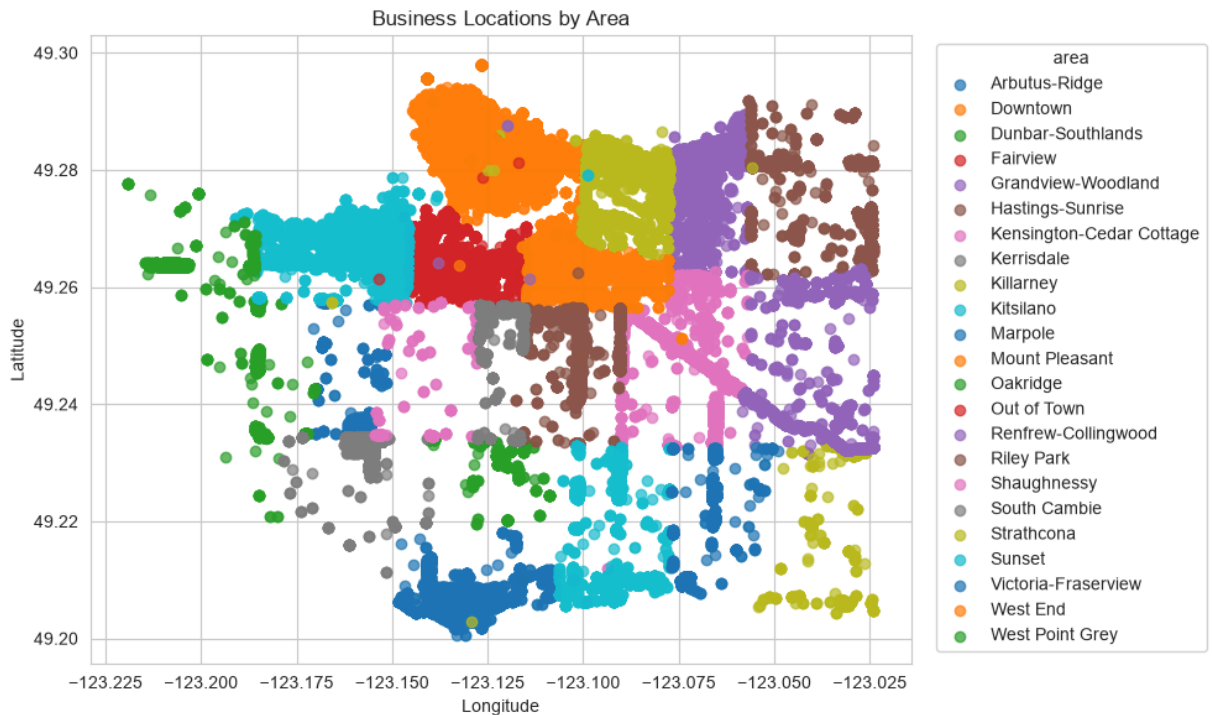
A2: Location-only clustering: K-means vs. DBSCAN

Initial Scatterplot Look

```
In [39]: # create scatter plot of Longitude vs Latitude
# group data by Neighborhood and color each group differently
# add labels, legend, and show plot

plt.figure(figsize=(10, 6))
for area, group in df.groupby("localarea"):
    lons = group["Longitude"]
    lats = group["Latitude"]
    plt.scatter(lons, lats, label=area, alpha=0.7)

plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("Business Locations by Area")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left", title="area")
plt.tight_layout()
plt.show()
```



Scaling: Geographic Features

```
In [40]: geo_features = df[["Longitude", "Latitude"]]
scaler_geo = StandardScaler()
geo_scaled = scaler_geo.fit_transform(geo_features)

print("Before scaling:")
print("Mean:\n", geo_features.mean())
print("Std:\n", geo_features.std())

print("\nAfter scaling:")
print("Mean:", geo_scaled.mean(axis=0))
print("Std:", geo_scaled.std(axis=0))
```

Before scaling:

Mean:

Longitude -123.112167

Latitude 49.265654

dtype: float64

Std:

Longitude 0.032212

Latitude 0.021541

dtype: float64

After scaling:

Mean: [-9.80718754e-14 2.85512378e-14]

Std: [1. 1.]

KMeans Clustering

```
In [41]: def kmeans_step_by_step(X, k, n_iterations, seed):
rng = np.random.default_rng(seed)
```

```

initial_idx = rng.choice(len(X), size=k, replace=False)
centroids = X[initial_idx]
history = [centroids.copy()]

labels = None
for _ in range(n_iterations):
    distances = np.linalg.norm(X[:, np.newaxis, :] - centroids[np.newaxis, :, :])
    labels = np.argmin(distances, axis=1)

    new_centroids = np.array([
        X[labels == i].mean(axis=0) if np.any(labels == i) else centroids[i]
        for i in range(k)
    ])
    centroids = new_centroids
    history.append(centroids.copy())

return history, labels

centroid_history, final_labels = kmeans_step_by_step(geo_scaled, k=4, n_iter=100)
print(len(centroid_history))

```

6

Graphing KMeans Clusters

```

In [42]: fig, axes = plt.subplots(2, 3, figsize=(15, 9))
axes = axes.flatten()

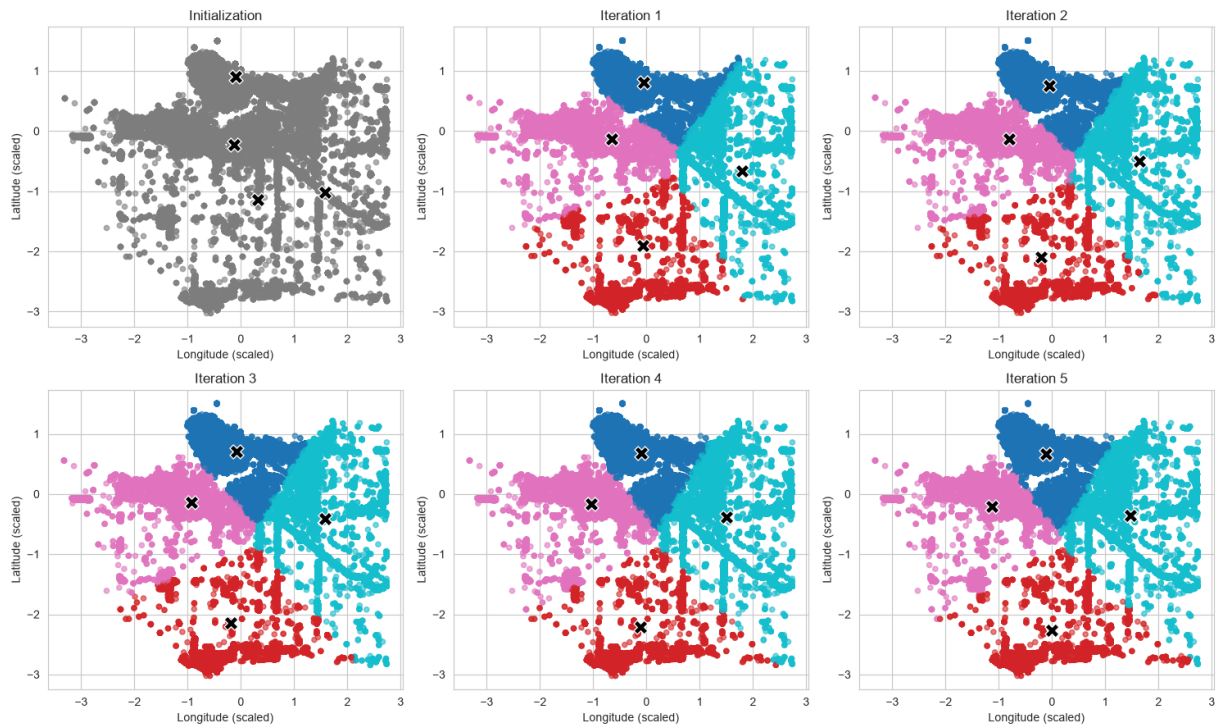
for i, ax in enumerate(axes):
    centroids = centroid_history[i]

    if i == 0:
        ax.scatter(geo_scaled[:, 0], geo_scaled[:, 1], c="gray", alpha=0.6, s=15)
        ax.set_title("Initialization")
    else:
        distances = np.linalg.norm(geo_scaled[:, np.newaxis, :] - centroids[np.newaxis, :, :])
        labels = np.argmin(distances, axis=1)
        ax.scatter(geo_scaled[:, 0], geo_scaled[:, 1], c=labels, cmap="tab10", s=15)
        ax.set_title(f"Iteration {i}")

    ax.scatter(centroids[:, 0], centroids[:, 1], c="black", marker="X", s=15)
    ax.set_xlabel("Longitude (scaled)")
    ax.set_ylabel("Latitude (scaled)")

plt.tight_layout()
plt.show()

```



Elbow Method & Silhouette Score

I took a 20% sample size for silhouette score to help speed up the process.

```
In [43]: inertias = []
k_max = 20
for k in range(1, k_max):
    km = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
    km.fit(geo_scaled)
    inertias.append(km.inertia_)

# Sample 20% of the dataset
X_subset = resample(geo_scaled, n_samples=int(0.2 * len(geo_scaled)), random

silhouette_scores = {}
for k in range(3, k_max):
    km = KMeans(n_clusters = k, random_state=RANDOM_STATE, n_init=10)
    km_labels = km.fit_predict(X_subset)
    silhouette_scores[k] = silhouette_score(X_subset, km_labels)

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))

ax[0].plot(range(1, k_max), inertias, marker="o")
ax[0].set_xlabel("Number of clusters (k)")
ax[0].set_ylabel("Inertia")
ax[0].set_title("Elbow Method")
ax[0].axvline(x=6, color="red", linestyle="--", label="k=6")
ax[0].legend()

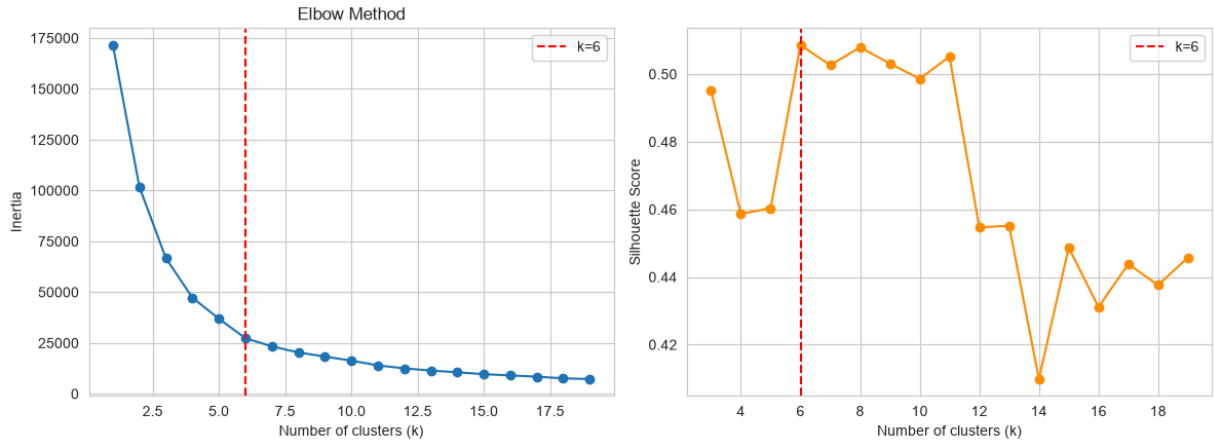
ax[1].plot(list(silhouette_scores.keys()), list(silhouette_scores.values()),
```

```

ax[1].set_xlabel("Number of clusters (k)")
ax[1].set_ylabel("Silhouette Score")
ax[1].axvline(x=6, color="red", linestyle="--", label="k=6")
ax[1].legend()

plt.tight_layout()
plt.show()

```



Looking at the graphs, I identify k=6 as the optimal cluster number.

Plotting KMeans with Optimal K

Here, I plot KMeans again with the optimal cluster number of k=6

```

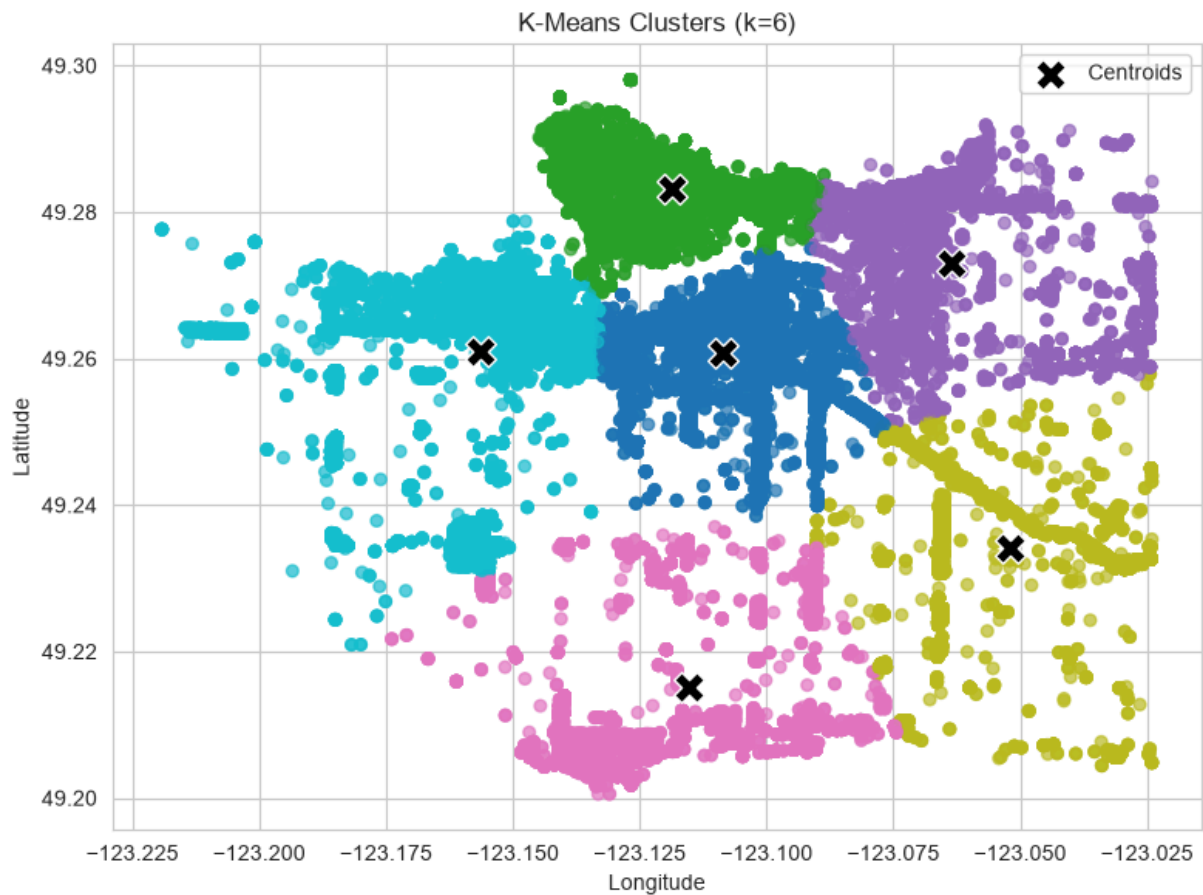
In [44]: kmeans_final = KMeans(n_clusters=6, random_state=RANDOM_STATE, n_init=10)
df["KMeans_Cluster"] = kmeans_final.fit_predict(geo_scaled)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(df["Longitude"], df["Latitude"], c=df["KMeans_Cluster"])

centroids_geo = scaler_geo.inverse_transform(kmeans_final.cluster_centers_)
plt.scatter(centroids_geo[:, 0], centroids_geo[:, 1], c="black", marker="X",

plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("K-Means Clusters (k=6)")
plt.legend()
plt.tight_layout()
plt.show()

```



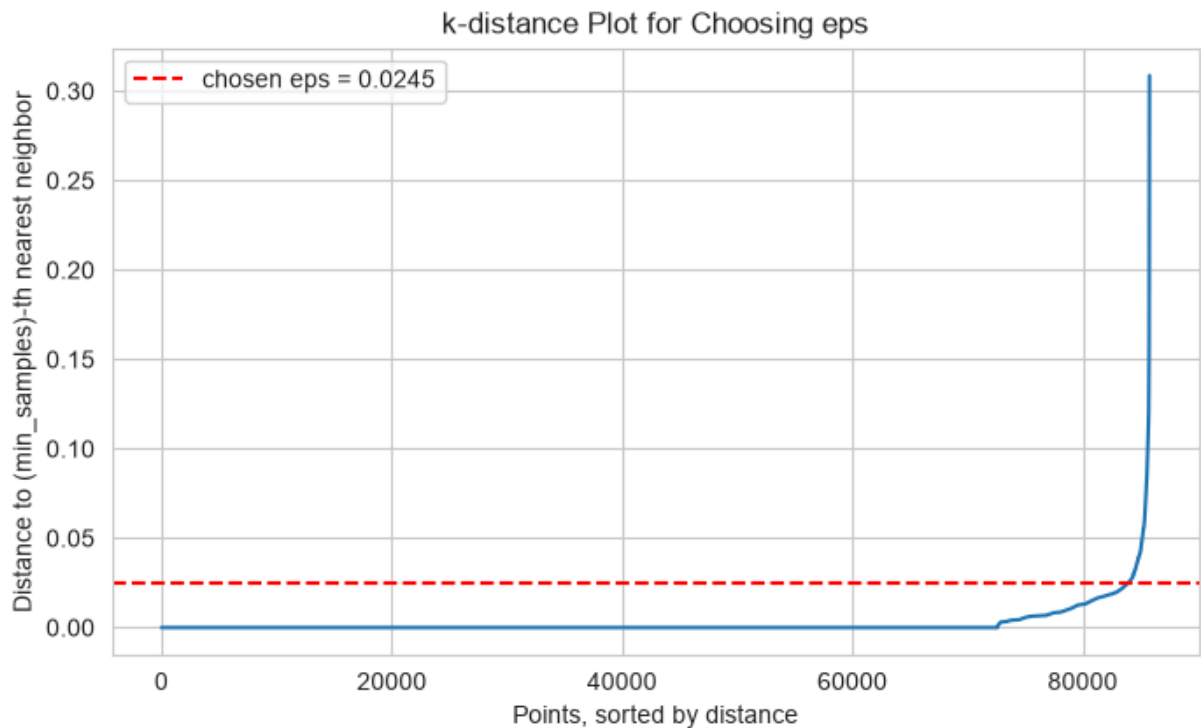
Evaluating: DBSCAN

```
In [45]: min_samples = 4

neighbours_model = NearestNeighbors(n_neighbors=min_samples)
neighbours_model.fit(geo_scaled)
distances, _ = neighbours_model.kneighbors(geo_scaled)

k_distances = np.sort_complex(distances[:, -1])

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(k_distances)
ax.axhline(0.0245, color="red", linestyle="--", label="chosen eps = 0.0245")
ax.set_xlabel("Points, sorted by distance")
ax.set_ylabel(f"Distance to (min_samples)-th nearest neighbor")
ax.set_title("k-distance Plot for Choosing eps")
ax.legend()
plt.show()
```



```
In [46]: dbscan = DBSCAN(eps=0.0245, min_samples=4)
df["DBSCAN_Cluster"] = dbscan.fit_predict(geo_scaled)

n_clusters_dbscan = len(set(df["DBSCAN_Cluster"])) - (1 if -1 in df["DBSCAN_

n_noise_dbscan = (df["DBSCAN_Cluster"] == -1).sum()
print(f"Number of clusters: {n_clusters_dbscan} and number of noise points:
```

Number of clusters: 1167 and number of noise points: 1922

I observe that my chosen parameters yields the following: Number of clusters: 1167 and number of noise points: 1922. I find these numbers & results to be surprising but also suspicious. Going forward, I will investigate why there is such a large discrepancy between the results yielded by KMeans and DBSCAN.

Investigating the Cluster-Count Discrepancy

As mentioned earlier, I argue that 1167 clusters is suspiciously high compared to the KMeans result of $k=6$, so I decided it's worth digging into why before trusting either number. KMeans always returns exactly k clusters by construction (it's not looking for density, it's minimizing variance), so the two counts were never guaranteed to agree. But 1167 clusters and 1922 noise points still seems too fragmented for a city-wide business dataset, so I decided to check for a concrete data issue: duplicate coordinates.

Many businesses (in malls, office towers, and other multi-tenant buildings) share the exact same `geo_point_2d`. If a large fraction of rows are literal duplicates, most points' distance to their `min_samples`-th nearest neighbor will be ~ 0 , which drags the

k-distance elbow down to a tiny `eps` that only bridges duplicates within the same building — too small to bridge to the building next door.

```
In [47]: coord_counts = df.groupby(["Longitude", "Latitude"]).size()

print(f"Total rows: {len(df)}")
print(f"Unique (Longitude, Latitude) pairs: {len(coord_counts)}")
print(f"Rows sharing a coordinate with at least one other row: {coord_counts

print("\nMost duplicated locations:")
print(coord_counts.sort_values(ascending=False).head())
```

Total rows: 85795

Unique (Longitude, Latitude) pairs: 8725

Rows sharing a coordinate with at least one other row: 85227

Most duplicated locations:

Longitude	Latitude	
-123.118825	49.263938	596
-123.118287	49.285718	537
-123.117751	49.283401	505
-123.121464	49.285245	485
-123.117898	49.281832	482

dtype: int64

After de-duplicating the locations, I observe that 85,795 rows collapse to just 8,725 unique locations. Moreover, 99% (85227 out of 85795) of rows share their coordinate with another row, and the busiest single address has nearly 600 businesses registered at it. It's also worth noting that because I dropped the fields containing business names,

This confirms my hypothesis: the original `eps=0.0245` was tuned almost entirely by these duplicate stacks, not by the spacing between distinct addresses.

Let's de-duplicate down to one row per unique coordinate and redo the k-distance plot on that — this shows the real spacing between distinct business locations, uncontaminated by the duplicate stacks.

```
In [48]: geo_unique = df[["Longitude", "Latitude"]].drop_duplicates().to_numpy()
geo_unique_scaled = scaler_geo.transform(geo_unique)

print(f"De-duplicated from {len(df)} rows down to {len(geo_unique)} unique l

neighbours_model_dedup = NearestNeighbors(n_neighbors=min_samples)
neighbours_model_dedup.fit(geo_unique_scaled)
distances_dedup, _ = neighbours_model_dedup.kneighbors(geo_unique_scaled)

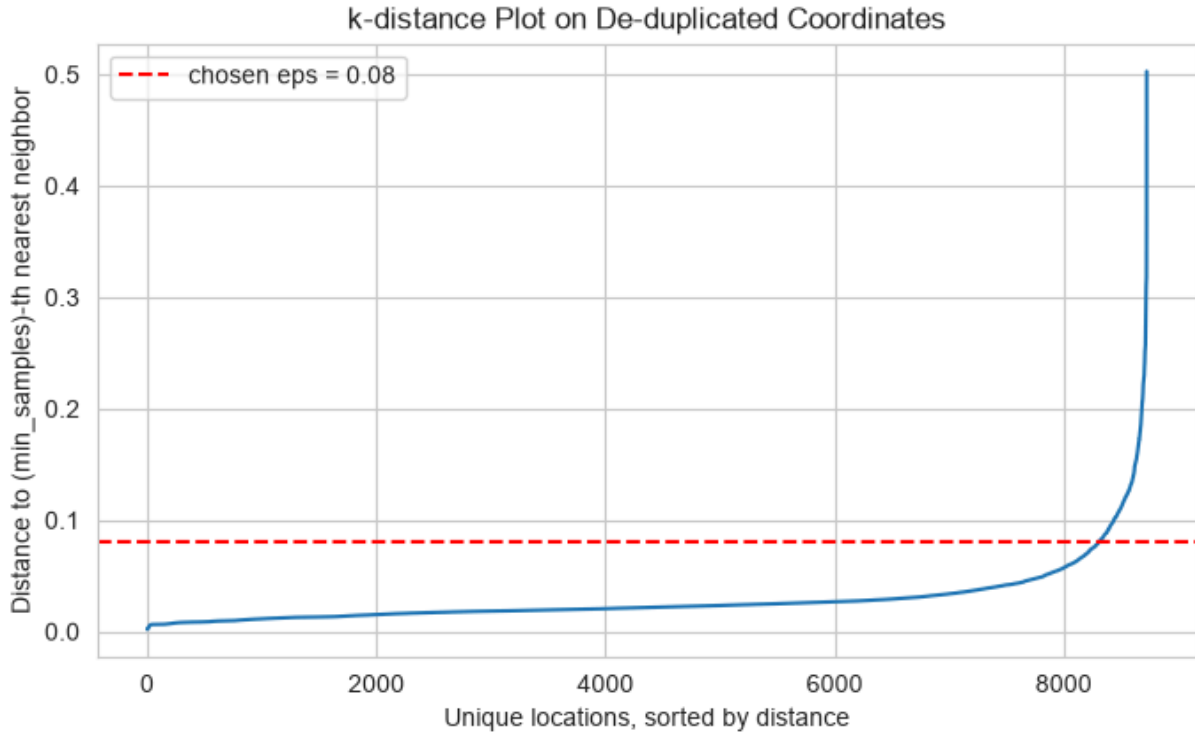
k_distances_dedup = np.sort(distances_dedup[:, -1])

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(k_distances_dedup)
ax.axhline(0.08, color="red", linestyle="--", label="chosen eps = 0.08")
ax.set_xlabel("Unique locations, sorted by distance")
ax.set_ylabel(f"Distance to (min_samples)-th nearest neighbor")
```



```
ax.set_title("k-distance Plot on De-duplicated Coordinates")
ax.legend()
plt.show()
```

De-duplicated from 85795 rows down to 8725 unique locations



Once the duplicate stacks are removed, the elbow sits much further out — around `eps=0.08` instead of `0.0245` — reflecting the real spacing between distinct addresses rather than the near-zero spacing between businesses sharing a building. Refit DBSCAN on the de-duplicated locations with this larger `eps`, then map each unique location's label back onto every row that shares its coordinate.

```
In [49]: dbscan_dedup = DBSCAN(eps=0.08, min_samples=min_samples)
dedup_labels = dbscan_dedup.fit_predict(geo_unique_scaled)

dedup_lookup = pd.DataFrame(geo_unique, columns=["Longitude", "Latitude"])
dedup_lookup["DBSCAN_Cluster_v2"] = dedup_labels

df = df.merge(dedup_lookup, on=["Longitude", "Latitude"], how="left")

n_clusters_v2 = len(set(dedup_labels)) - (1 if -1 in dedup_labels else 0)
n_noise_v2 = (df["DBSCAN_Cluster_v2"] == -1).sum()
print(f"Number of clusters: {n_clusters_v2} and number of noise points: {n_r
```

Number of clusters: 55 and number of noise points: 1377 (out of 85795 rows)

```
In [50]: fig, ax = plt.subplots(1, 2, figsize=(14,7))

noise_mask = df["DBSCAN_Cluster"] == -1
ax[0].scatter(
    df.loc[~noise_mask, "Longitude"],
    df.loc[~noise_mask, "Latitude"],
    c = df.loc[~noise_mask, "DBSCAN_Cluster"],
```

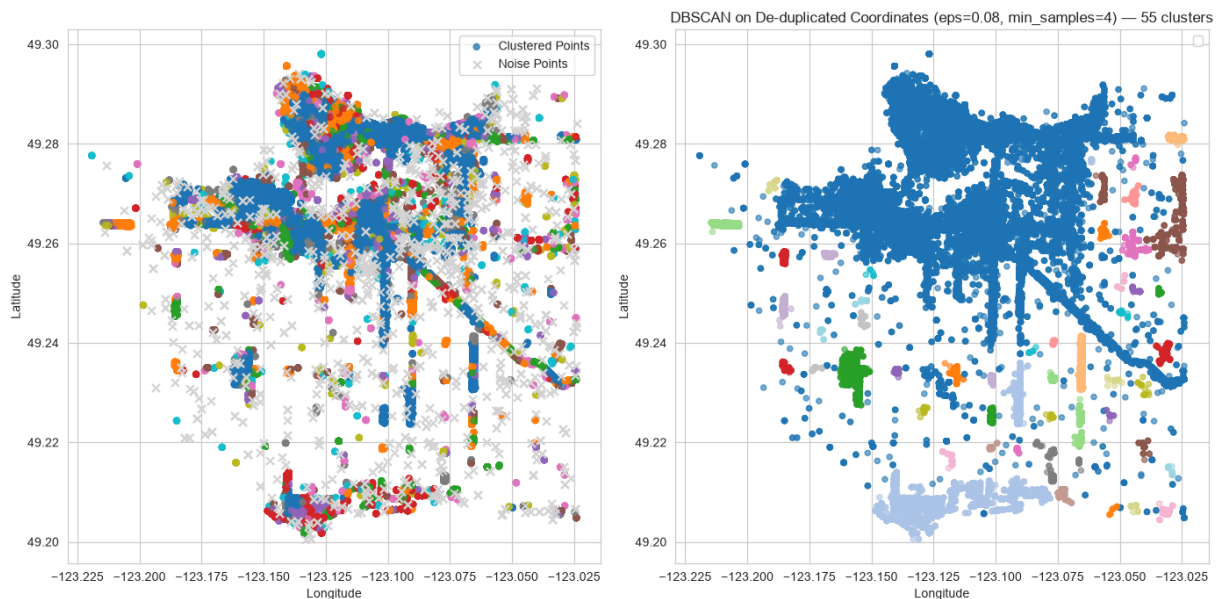
```

cmap="tab10",
s=25,
alpha=0.75,
label="Clustered Points"
)
ax[0].scatter(
    df.loc[noise_mask, "Longitude"],
    df.loc[noise_mask, "Latitude"],
    c = "lightgrey",
    marker="x",
    s=40,
    label="Noise Points"
)
ax[0].set_xlabel("Longitude")
ax[0].set_ylabel("Latitude")
ax[0].legend()

ax[1].scatter(
    df["Longitude"],
    df["Latitude"],
    c=df["DBSCAN_Cluster_v2"],
    cmap="tab20",
    alpha=0.6,
    s=20)

ax[1].set_xlabel("Longitude")
ax[1].set_ylabel("Latitude")
ax[1].set_title(f"DBSCAN on De-duplicated Coordinates (eps=0.08, min_samples=4) — 55 clusters")
ax[1].legend()
plt.tight_layout()
plt.show()

```



This gives ~55 clusters and only ~1,400 noise points (1.6% of rows) — a much more plausible result than the original 1167 clusters / 1922 noise points, and much closer with the ~22 official Vancouver local areas/neighbourhoods

(https://en.wikipedia.org/wiki/List_of_neighbourhoods_in_Vancouver) split into finer, distinct commercial nodes.

A2 - Write Up - Clustering

1. How do the K-means clusters map onto actual Vancouver geography — do they correspond to anything you'd recognize (downtown core, west side, etc.)?

The K-means clusters maps well onto actual Vancouver geography. I identified 6 main districts by using the elbow-method, which maps to generally "Downtown", "West-side (Kits/West-point Grey)", "Granville "Fairview/mount pleasant", "East-side (Strathcona/Granview)", "south-side "Marpole", and "south-east side "Kensington/Renfrew".

2. How do the DBSCAN clusters compare? Where do the two methods agree or disagree, and why — think about what each algorithm assumes about cluster shape and density, and what DBSCAN's "noise" points represent.

DBSCAN on the de-duplicated coordinates resolves to 55 clusters, a much finer partition than K-means' of 6. At a coarse level, both methods agree on where business activity concentrates. Indeed, every one of DBSCAN's 55 dense nodes falls inside the boundary of one of K-means' 6 broad regions rather than scattered randomly across them. For example, the cluster of DBSCAN nodes running along Kingsway (a dense commercial strip stretching toward Burnaby) all sit inside K-means' "East-side" partition, and the tight cluster of DBSCAN nodes downtown all sit inside K-means' "Downtown" partition. So the two methods aren't contradicting each other about broad geography, they're operating at different resolutions of the same underlying pattern. The disagreement is about granularity and about which points get a label at all, and that traces directly back to what each algorithm assumes. K-means assumes clusters are convex, roughly-equal-size regions defined purely by distance to a centroid — it partitions the entire map into exactly k Voronoi-like slices no matter how the underlying density actually looks, which is why it draws one broad "East-side" region instead of noticing that businesses inside that region are really packed tightly along Kingsway and much sparser elsewhere. DBSCAN makes no shape assumption at all; it only requires a minimum number of points (`min_samples`) within a radius (`eps`) to grow a cluster, so it can trace an irregular, elongated shape like a strip along a single road, and it can also refuse to assign a cluster label anywhere the density requirement isn't met. That's what DBSCAN's "noise" (-1) points represent: businesses whose local neighbourhood is too sparse to be density-connected to any commercial node — i.e. a single storefront on an otherwise residential block, or an address on the edge of the city. K-means has no equivalent concept; every point is forced into one of the 6 centroids regardless of how isolated it actually is. In this dataset only ~1,377 of 85,795 rows (~1.6%) end up as noise post-deduplication, so the vast majority of licensed businesses do sit within a recognizable commercial node — it's really only the genuine outliers that DBSCAN treats differently from K-means.

A3: Feature-based clustering: Size, Industry, Lifecycle

Engineering Features

This section moves away from geography and clusters businesses on three attribute groups instead:

- **Size** — `numberofemployees` and `feepaid`
- **Industry** — the consolidated `industry` label built in A1 (`businesstype` + `businesssubtype`), encoded
- **Lifecycle** — licence duration (`expireddate` - `issueddate`), the month the licence was issued (cyclically encoded), and `licencerevisionnumber`

I'll build each group separately, scale everything onto a comparable footing, then combine into one feature matrix for K-means.

Lifecycle Features

`issueddate` and `expireddate` are both parsed as dates so I can derive:

- `licence_duration_days` = `expireddate` - `issueddate`
- `month_issued_sin` / `month_issued_cos` — the calendar month a licence was issued, cyclically encoded so December and January stay close together instead of 11 apart
- `licence_revision_number` — `licencerevisionnumber` cast to an integer (it's zero-padded text in the source data, e.g. `"00"`, `"01"`, `"10"`)

A small number of rows (~0.2%) are missing `issueddate` / `expireddate` entirely and get dropped.

```
In [51]: n_before = len(df)

df["issueddate_dt"] = pd.to_datetime(df["issueddate"], utc=True, errors="coerce")
df["expireddate_dt"] = pd.to_datetime(df["expireddate"], utc=True, errors="coerce")
df = df.dropna(subset=["issueddate_dt", "expireddate_dt"]).copy()

print(f"Dropped {n_before - len(df)} rows missing issueddate/expiredate")

df["licence_duration_days"] = (df["expireddate_dt"] - df["issueddate_dt"]).dt.days

month_issued = df["issueddate_dt"].dt.month
df["month_issued_sin"] = np.sin(2 * np.pi * month_issued / 12)
df["month_issued_cos"] = np.cos(2 * np.pi * month_issued / 12)

df["licence_revision_number"] = df["licencerevisionnumber"].astype(int)
```

```
df["licence_duration_days"].describe()
```

Dropped 145 rows missing issueddate/expireddate

```
Out[51]: count      85650.000000
         mean        350.331909
         std         81.313304
         min        -354.000000
         25%        345.000000
         50%        377.000000
         75%        399.000000
         max         601.000000
         Name: licence_duration_days, dtype: float64
```

A small number of rows (66, ~0.08%) come out with a *negative* duration — the licence's `expireddate` falls before its `issueddate`. This looks like a data-entry quirk around renewals (e.g. a licence renewed late, where the new `issueddate` is stamped after the old `expireddate` it's replacing) rather than a real business attribute, so these rows are dropped.

```
In [52]: n_before = len(df)
         df = df[df["licence_duration_days"] >= 0].copy()
         print(f"Dropped {n_before - len(df)} rows with a negative licence duration")
         print(f"Remaining rows: {len(df)}")
```

Dropped 66 rows with a negative licence duration
Remaining rows: 85584

Size Features

`numberofemployees` and `feepaid` are both heavily right-skewed (a handful of businesses report thousands of employees or tens of thousands in fees), so I log-transform both (`log1p`, which handles the `numberofemployees == 0` case) before scaling.

`feepaid` is also missing on ~31% of rows. Rather than guess at *why* a fee wasn't recorded, I impute the missing values with the column median and add a `fee_missing` flag so K-means can still separate "actually cheap" from "unknown fee" if that turns out to matter.

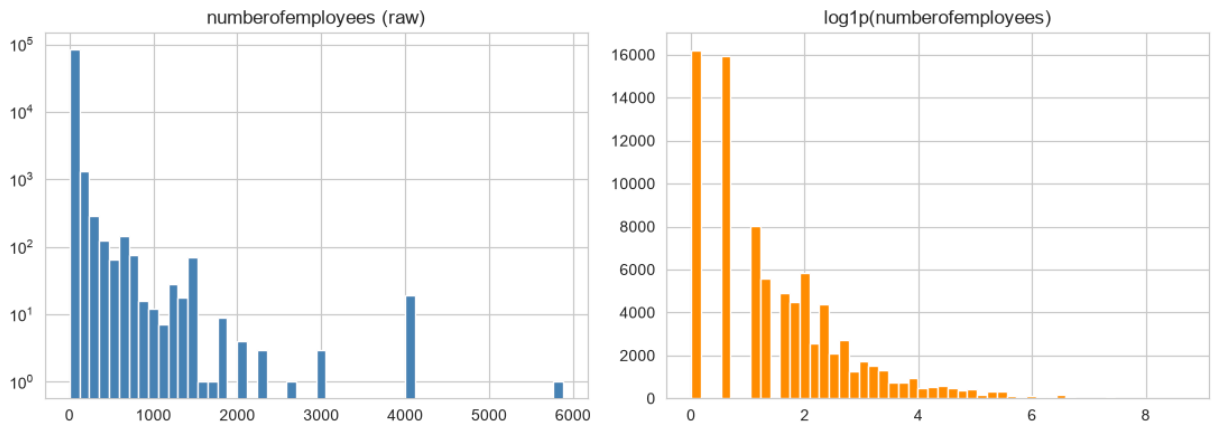
```
In [53]: df["fee_missing"] = df["feepaid"].isna().astype(int)
         df["feepaid_imputed"] = df["feepaid"].fillna(df["feepaid"].median())

         df["log_employees"] = np.log1p(df["numberofemployees"])
         df["log_feepaid"] = np.log1p(df["feepaid_imputed"])

         fig, axes = plt.subplots(1, 2, figsize=(11, 4))
         axes[0].hist(df["numberofemployees"], bins=50, color="steelblue")
         axes[0].set_yscale("log")
         axes[0].set_title("numberofemployees (raw)")
         axes[1].hist(df["log_employees"], bins=50, color="darkorange")
         axes[1].set_title("log1p(numberofemployees)")
```

```
plt.tight_layout()
plt.show()
```

```
df[["numberofemployees", "feepaid_imputed", "log_employees", "log_feepaid",
```



Out [53]:

	numberofemployees	feepaid_imputed	log_employees	log_feepaid	fee_mi
count	85584.000000	85584.000000	85584.000000	85584.000000	85584.000000
mean	18.031022	541.532648	1.565459	5.773236	0.301022
std	103.353251	1274.723873	1.308556	0.920995	0.460517
min	0.000000	2.000000	0.000000	1.098612	0.000000
25%	1.000000	277.000000	0.693147	5.627621	0.000000
50%	3.000000	277.000000	1.386294	5.627621	0.000000
75%	9.000000	331.000000	2.302585	5.805135	1.000000
max	5876.000000	63722.000000	8.678802	11.062301	1.000000

Industry Feature

The `industry` column from A1 (`businessstype` + `businesssubtype` where a subtype exists) has 156 unique values, and one-hot encoding all of them would let industry dominate the distance metric relative to the handful of size/lifecycle columns, while also giving K-means 156 mostly-empty, high-cardinality dimensions to fit noise on.

Instead, I keep the 20 most common `industry` values as their own categories and collapse everything else into `"Other"`, then one-hot encode that consolidated column (21 dimensions total).

In [54]:

```
TOP_N_INDUSTRIES = 20
```

```
top_industries = df["industry"].value_counts().nlargest(TOP_N_INDUSTRIES).index
df["industry_consolidated"] = np.where(df["industry"].isin(top_industries),
industry_dummies = pd.get_dummies(df["industry_consolidated"], prefix="indus
```

```
print(f"Consolidated {df['industry'].nunique()} industry values down to {df['industry_consolidated'].value_counts()")
```

Consolidated 151 industry values down to 21 categories

```
Out[54]: industry_consolidated
Other 22655
Health Care Professionals and Services 11771
Long-term Rental 9392
Retail Dealer 6007
Legal Services 5199
Restaurant – Class 1 with liquor service 3334
Business Support Services 2957
Financial Services 2398
Consulting and Management Services 2351
Limited Service Food Establishment – Without Liquor 2343
Real Estate Services 2049
Retail Dealer – Food 1935
General Contractor 1785
Information Communication Technology 1702
Association or Society 1611
Restaurant – Class 1 no liquor service 1503
Wholesale Dealer – Non-Food 1454
Limited Service Food Establishment 1428
Beauty Services – Barber Shop or Beauty Salon 1354
Beauty Services – Beauty and Wellness Centre 1318
Health Care Facility 1038
Name: count, dtype: int64
```

Combining & Scaling the Feature Set

I then apply `StandardScaler` to the continuous numeric columns (`log_employees` , `log_feepaid` , `licence_duration_days` , `licence_revision_number`) so none of them dominates purely because of its raw units. The cyclical month features (`month_issued_sin/cos`) are already bounded to `[-1, 1]` and the `industry/ fee_missing` columns are already binary 0/1, so both are left as-is — scaling them further would only distort the meaning of "same category" vs. "different category."

```
In [55]: continuous_cols = ["log_employees", "log_feepaid", "licence_duration_days",

scaler_features = StandardScaler()
continuous_scaled = pd.DataFrame(
    scaler_features.fit_transform(df[continuous_cols]),
    columns=continuous_cols,
    index=df.index,
)

feature_matrix = pd.concat(
    [continuous_scaled, df[["month_issued_sin", "month_issued_cos", "fee_mis
    axis=1,
)

X_features = feature_matrix.to_numpy()
```

```
print(f"Feature matrix shape: {feature_matrix.shape}")
feature_matrix.head()
```

Feature matrix shape: (85584, 28)

Out[55]:

	log_employees	log_feepaid	licence_duration_days	licence_revision_number	mont
0	-1.196332	-0.378165	0.699333	-0.683942	
1	-1.196332	-0.165946	0.699333	-0.683942	
2	-1.196332	-0.577085	0.004594	-0.683942	
3	-1.196332	0.335490	-0.342775	-0.683942	
4	-1.196332	-0.138752	0.612490	-0.683942	

5 rows x 28 columns

```
In [56]: numeric_features = feature_matrix.select_dtypes(include='number').columns.to
corr = feature_matrix[numeric_features].corr()

# Plot as heatmap
fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(
    corr,
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    center=0,
    linewidths=0.5,
    linecolor='white',
    ax=ax,
    annot_kws={'size': 8}
)
plt.tight_layout()
plt.show()
```




K-means Clustering

Here, I'll be using the same approach as A2: an elbow plot over inertia, plus a silhouette score computed on a 20% sample for speed.

```
In [57]: k_max = 12

inertias = []
for k in range(1, k_max):
    km = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
    km.fit(X_features)
    inertias.append(km.inertia_)

X_features_subset = resample(X_features, n_samples=int(0.2 * len(X_features)))
silhouette_scores = {}
for k in range(2, k_max):
    km = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
    km_labels = km.fit_predict(X_features_subset)
    silhouette_scores[k] = silhouette_score(X_features_subset, km_labels)

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
```

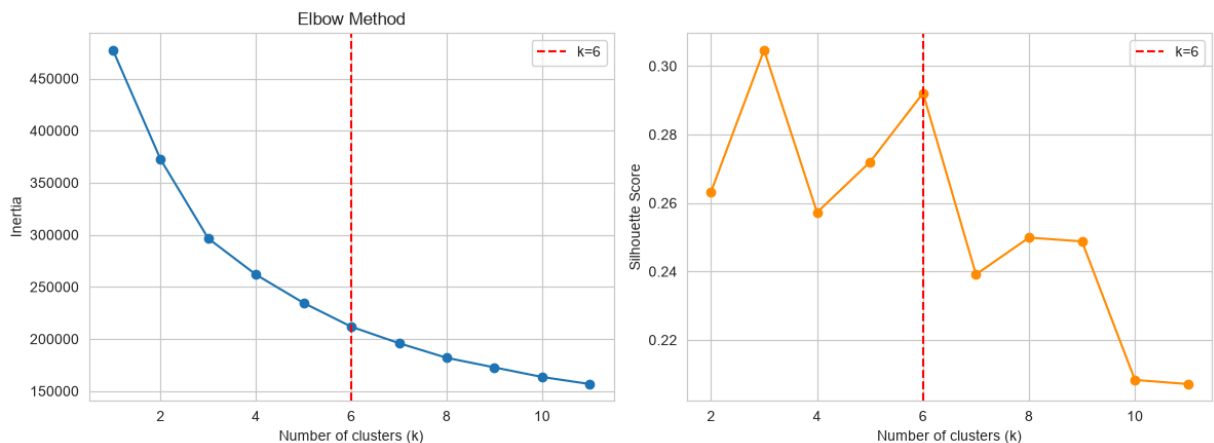
```

ax[0].plot(range(1, k_max), inertias, marker="o")
ax[0].set_xlabel("Number of clusters (k)")
ax[0].set_ylabel("Inertia")
ax[0].set_title("Elbow Method")
ax[0].axvline(x=6, color="red", linestyle="--", label="k=6")
ax[0].legend()

ax[1].plot(list(silhouette_scores.keys()), list(silhouette_scores.values()),
ax[1].set_xlabel("Number of clusters (k)")
ax[1].set_ylabel("Silhouette Score")
ax[1].axvline(x=6, color="red", linestyle="--", label="k=6")
ax[1].legend()

plt.tight_layout()
plt.show()

```



The elbow is gradual rather than sharp — expected, since a mix of continuous and one-hot features rarely produces the clean knee a single spatial feature pair does. Silhouette peaks slightly at $k=3$ (0.30) but $k=6$ (0.29) is a close second and, when I inspect cluster means, $k=3$ barely separates on size/lifecycle (employees ~15-20, duration ~370-375 days across all three groups) while $k=6$ splits out clearly distinct profiles — e.g. one cluster of large employers (~100 employees), one of short-duration licences (~140 days), and one of high-fee licences (~\$2,900). I'll go with **$k=6$** , trading a marginally lower silhouette score for clusters that are actually interpretable, and it keeps this section comparable to the $k=6$ chosen for location in A2.

```

In [58]: kmeans_features_final = KMeans(n_clusters=6, random_state=RANDOM_STATE, n_iter=1000)
df["Feature_Cluster"] = kmeans_features_final.fit_predict(X_features)

df["Feature_Cluster"].value_counts().sort_index()

```

```

Out[58]: Feature_Cluster
0      6878
1      1687
2     33036
3     24809
4      11015
5       8159
Name: count, dtype: int64

```

Cluster Profiles

A quick sanity check on what each cluster actually represents, before projecting to 2D.

```
In [59]: profile = df.groupby("Feature_Cluster")[["numberofemployees", "feepaid_imputed", "licence_duration_days", "top_industry"]
profile["top_industry"] = df.groupby("Feature_Cluster")["industry_consolidated"].size()
profile["n_businesses"] = df.groupby("Feature_Cluster").size()
profile
```

```
Out[59]:
```

	numberofemployees	feepaid_imputed	licence_duration_days	top_industry
Feature_Cluster				
0	5.549142	2919.889309	359.804304	Long
1	21.780083	13.709544	349.512152	Assoc or S
2	2.943244	333.135573	374.442124	
3	6.687049	277.158128	372.780201	
4	101.712846	565.809714	380.040218	
5	10.388283	260.635699	139.650815	

PCA Projection to 2D

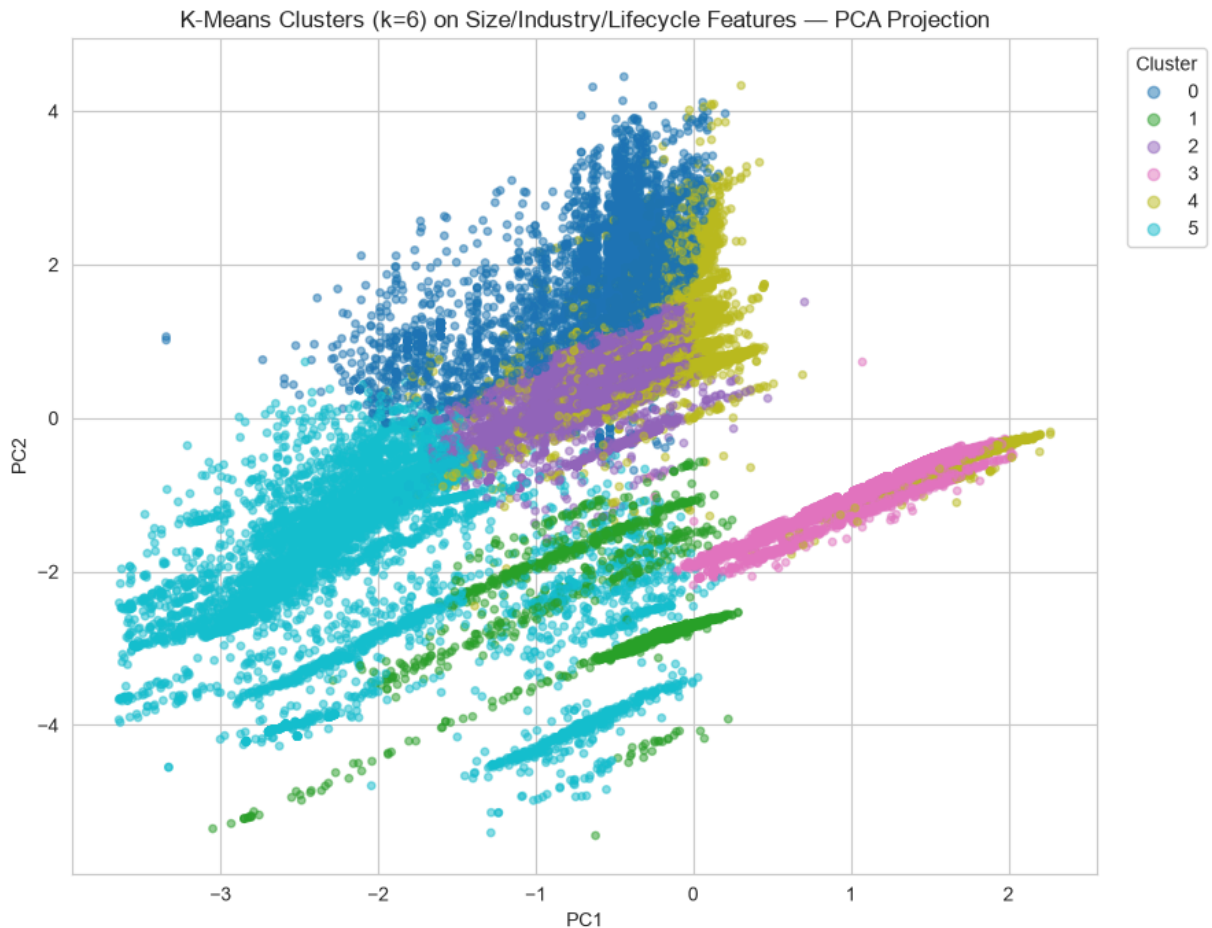
The feature matrix has 28 dimensions (4 scaled numeric + 2 cyclical + `fee_missing` + 21 industry dummies), too many to plot directly. PCA compresses it down to the 2 directions of greatest variance so the clusters can be visualized on a scatter plot.

```
In [60]: pca_features = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca = pca_features.fit_transform(X_features)

print(f"Explained variance ratio: PC1={pca_features.explained_variance_ratio_[0]}, PC2={pca_features.explained_variance_ratio_[1]}")
print(f"Total variance captured: {pca_features.explained_variance_ratio_.sum()}")

plt.figure(figsize=(9, 7))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df["Feature_Cluster"], cmap=cm.tab10)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("K-Means Clusters (k=6) on Size/Industry/Lifecycle Features – PCA")
plt.legend(*scatter.legend_elements(), title="Cluster", bbox_to_anchor=(1.02, 1))
plt.tight_layout()
plt.show()
```

Explained variance ratio: PC1=0.233, PC2=0.228
Total variance captured: 0.461



A3 - Write Up - PCA Clustering

1. What do the resulting clusters seem to represent? Does this differ from the purely geographic clustering in A2?

The two components capture ~46% of total variance — a reasonable amount given the feature set mixes continuous lifecycle/size signals with a wide, sparse block of industry dummies (which individually contribute very little variance each). The dominant loadings on PC1/PC2 are `licence_revision_number` and `licence_duration_days`, so the 2D PCA view is mostly a "lifecycle" projection. But the PCA plot is only a compressed view of a 28-dimensional clustering — looking at the full cluster profile table (which is built from the actual features, not the 2D projection) shows each of the 6 clusters is really anchored by a *different* one of size, industry, or lifecycle, not lifecycle alone:

- **Cluster 0** (n=6,878) — anchored in `industry`: top type is Long-term Rental, and it carries a distinctly high average fee (~\$2,920 vs. ~260–570 in every other cluster), consistent with rental licence fees scaling with number of units.
- **Cluster 1** (n=1,687) — also anchored in `industry`: top type is Association or Society, with a distinctly high average employee count (~21.8) but a near-zero average fee (~\$13.7), consistent with membership/non-profit organizations that report staff but pay little to no licence fee.

- **Cluster 4** (n=11,015) — anchored in **size** : average employees (~101.7) dwarfs every other cluster (2.9–21.8 elsewhere) — these are the large employers, regardless of industry.
- **Cluster 5** (n=8,159) — anchored in **lifecycle** : average licence duration (~139.7 days) is far shorter than the ~350–380 days everywhere else — likely newly-issued or otherwise short-term licences.
- **Clusters 2 and 3** (n=33,036 and 24,809, the two largest) — both top out at "Other" industry and sit close to the corpus average on employees, fee, *and* duration, so they're best read as the undifferentiated bulk of ordinary small businesses. Their only real point of separation is a small residual difference in duration (374.4 vs. 372.8 days), which is exactly what PC1/PC2's lifecycle framing is picking up on — for these two clusters specifically, lifecycle timing is the only axis left to split on once size and industry are already unremarkable.

So the earlier read of "lifecycle dominates" only really holds for the two large, otherwise-generic clusters; the four smaller clusters are each driven by whichever of size, industry, or lifecycle is most extreme for that group of businesses.

This differs substantially from the purely geographic clustering in A2. None of the size/industry/lifecycle features encode location, so these clusters cut completely across geography: Cluster 0 (rental-heavy, high-fee) and Cluster 4 (large employers) each contain businesses scattered across every neighbourhood in the city, not concentrated in one area the way A2's clusters were by construction. A2 grouped businesses by where they physically sit; A3 groups them by what kind of business they are and how they operate — a rental business in Kitsilano and a rental business in Renfrew-Collingwood land in the same A3 cluster despite being in entirely different A2 clusters.

Part B

This section will be where I put in my part B answers just for reference. They will be present in the Streamlit app too.

B4 - Write Up - Clustering

1. Does the grouping seem meaningful given what you know (or can look up) about these areas? Any surprising groupings? Explain using the actual cluster membership, not just the general shape of the plot

Regarding meaningful groupings, I observe that Cluster 5 (singleton containing Downtown) is rather accurate. Downtown is Vancouver's core financial/legal district, so a profile topped by legal, business-support, and consulting services standing apart from every residential neighbourhood makes sense to me. In the same way, Cluster 0's wholesale/non-food signature fits the local areas well. Mount Pleasant, Grandview-

Woodland, and Strathcona are old warehouse and light-industrial eastside areas that are now mixed with some rental apartments, and Marpole has its own industrial strip along the Fraser. Cluster 4 contains Sunset and Hastings-Sunrise, which are similar working-class East Van neighbourhoods with comparable commercial strips. Cluster 2's sky-high long-term rental share fits Kitsilano and the West End very well. The West End is generally Vancouver's more densely populated area, with large rental apartment blocks

What is surprising to me, however, is that Shaughnessy is landing in the more rental-heavy Cluster 2. Shaughnessy is known to be one of the more affluent/wealthy areas with lots of heritage mansions and large properties, which is very far from an apartment-rental market. I argue this may be due to a small-sample artifact. I observe that Shaughnessy has only 355 registered businesses (the smallest of any area in the dataset), so a handful of rental-related or property-management listings can potentially swing its percentage profile enough to statistically resemble West-End/Kits, even though the underlying neighbourhood is very different. I also found Fairview anchoring the more "health-care" cluster of cluster 3 to be an interesting observation. Broadway in Fairview is a major employment hub (as demonstrated with their 10k+ registered businesses in the dataset, second only to Downtown), but it's also a hub that's built around health care facilities such as Vancouver General Hospital, BC Cancer Agency, and the BC Centre for Disease Control. As such, the fellow areas in cluster 3 (Renfrew, Victoria-Fraserview, Killarney, Dunbar, and Oakridge) don't have a comparable health care corridor. I argue that they likely land in the same cluster because they're residential areas with limited business diversity, where healthcare clinics and offices make up an outsized share of what commercial activity exists.

Ultimately, because clustering here is done on relative composition rather than absolute counts, small-business-count neighbourhoods (such as Shaughnessy and South Cambie) are just as influential as huge ones (Fairview and Downtown). I observe this is what lets a mansion district like Shaughnessy get grouped with West Van's densest rental zones, and a hospital district gets grouped with more quiet family suburbs (Renfrew, Killarney, etc.).